Status	Started	Completed
Finished	Wednesday, 11 November 2026, 1:08 PM	Wednesday, 11 November 2026, 1:09 PM

Duration	Grade
1 min 5 secs	**20.00** out of 20.00 (**100**%)

Question 1

Flag question

Correct Mark 1.00 out of 1.00

You want Claude Code to automatically run npm test after every TypeScript file edit. Where is the correct place to configure this automated test execution?

a. Create a .git/hooks/post-commit script that runs npm test.
b. Add run test after edits to the session's system prompt.
c. Configure a PostToolUse hook in the project CLAUDE.md that specifies the npm test command to run after every TypeScript file-edit tool call. Claude Code's hooks system is designed for post-action automation like this. ✓
d. Add a comment at the top of each TypeScript file: '// run npm test after editing'.

The correct answer is: Configure a PostToolUse hook in the project CLAUDE.md that specifies the npm test command to run after every TypeScript file-edit tool call. Claude Code's hooks system is designed for post-action automation like this.

Question 2

Flag question

Correct Mark 1.00 out of 1.00

A developer provides Claude Code with the instruction 'optimise this module for performance.' Claude Code significantly changes the module's public API, breaking downstream consumers. What development approach would have surfaced this risk before implementation?

a. Run tests after the changes to detect API breakage.
b. Always use Plan Mode for performance optimisation tasks.
c. Use the interview pattern before implementation: have Claude Code ask clarifying questions about acceptable interface changes, downstream consumers, performance benchmarks, and acceptable trade-offs. This surfaces hidden constraints like API compatibility before any changes are made. ✓
d. Provide a more detailed performance specification.

The correct answer is: Use the interview pattern before implementation: have Claude Code ask clarifying questions about acceptable interface changes, downstream consumers, performance benchmarks, and acceptable trade-offs. This surfaces hidden constraints like API compatibility before any changes are made.

Question 3

Flag question

Correct Mark 1.00 out of 1.00

You create a /document-api slash command in .claude/commands/document-api.md. A colleague clones the repository and reports that /document-api is not available in their Claude Code session. What is the most likely cause?

a. Slash commands created in .claude/commands/ are only available to the file's author.
b. The colleague needs to run /reload-commands to refresh the command list.
c. The colleague may be looking in ~/.claude/commands/ — commands in .claude/commands/ at the project level are version-controlled and auto-available. Verify the file exists at .claude/commands/document-api.md in the repository root and that the colleague pulled the latest changes. ✓
d. Slash commands require manual activation before use.

The correct answer is: The colleague may be looking in ~/.claude/commands/ — commands in .claude/commands/ at the project level are version-controlled and auto-available. Verify the file exists at .claude/commands/document-api.md in the repository root and that the colleague pulled the latest changes.

Question 4

Flag question

Correct Mark 1.00 out of 1.00

You want to create a skill that searches for all TODO comments in a codebase and generates a prioritised backlog. The skill should only read files, never modify them. It will produce verbose output during discovery. What is the correct SKILL.md frontmatter configuration?

a. Use context: fork only — allowed-tools defaults to all tools if not specified.
b. context: fork prevents main session pollution; allowed-tools: ['Read', 'Grep', 'Glob'] restricts to read-only operations; argument-hint: 'Repository root path' prompts for the directory. All three work together to create a safe, isolated, targeted skill. ✓
c. Use allowed-tools: ['Read'] only — Grep and Glob are not needed for file content search.
d. No frontmatter needed — system prompt instructions are sufficient.

The correct answer is: context: fork prevents main session pollution; allowed-tools: ['Read', 'Grep', 'Glob'] restricts to read-only operations; argument-hint: 'Repository root path' prompts for the directory. All three work together to create a safe, isolated, targeted skill.

Question 5

Flag question

Correct Mark 1.00 out of 1.00

Your team wants Claude Code to apply TypeScript-specific conventions only when editing .ts and .tsx files, and Python conventions only when editing .py files. Test files (*.test.ts) need testing-specific conventions regardless of their directory. Which configuration achieves all three automatically?

a. Create separate CLAUDE.md files in typescript/, python/, and tests/ directories.
b. Configure three separate Claude Code projects, one per technology stack.
c. Create .claude/rules/ files with YAML frontmatter path patterns: paths: ['**/*.ts', '**/*.tsx'] for TypeScript rules, paths: ['**/*.py'] for Python rules, and paths: ['**/*.test.ts', '**/*.spec.ts'] for testing rules. Glob patterns activate rules based on file path regardless of directory structure. ✓
d. Add all three convention sets to the root CLAUDE.md with headers marking each section.

The correct answer is: Create .claude/rules/ files with YAML frontmatter path patterns: paths: ['**/*.ts', '**/*.tsx'] for TypeScript rules, paths: ['**/*.py'] for Python rules, and paths: ['**/*.test.ts', '**/*.spec.ts'] for testing rules. Glob patterns activate rules based on file path regardless of directory structure.

Question 6

Flag question

Correct Mark 1.00 out of 1.00

Your team maintains API documentation standards in a file at ./docs/api-standards.md that is updated quarterly. You want Claude Code to always use the current version without requiring manual CLAUDE.md updates each quarter. What is the correct configuration?

a. Copy the api-standards.md content into CLAUDE.md at the start of each quarter.
b. Create a PostToolUse hook that injects the latest standards after each file edit.
c. Add @./docs/api-standards.md to your CLAUDE.md — the @import syntax loads the referenced file's content at session start, so quarterly updates to api-standards.md are automatically reflected in Claude Code without any changes to CLAUDE.md itself. ✓
d. Store the URL of the standards document in CLAUDE.md and fetch it at runtime.

The correct answer is: Add @./docs/api-standards.md to your CLAUDE.md — the @import syntax loads the referenced file's content at session start, so quarterly updates to api-standards.md are automatically reflected in Claude Code without any changes to CLAUDE.md itself.

Question 7

Flag question

Correct Mark 1.00 out of 1.00

Your organisation runs both a public-facing web application and an internal tool using the same Claude Code setup. The web application requires strict security review conventions; the internal tool has more relaxed requirements. How do you manage these different standards?

- a. Configure each application as a separate project with its own .claude/CLAUDE.md. The web application's project CLAUDE.md enforces strict security conventions; the internal tool's project CLAUDE.md has appropriate relaxed conventions. Project-level configuration scopes standards to their relevant context. ✓
- b. Use a single CLAUDE.md with a flag that developers toggle between strict and relaxed mode.
- c. Apply the strictest standards universally — the overhead is worth the consistency.
- d. Apply strict standards during business hours and relaxed standards after hours.

The correct answer is: Configure each application as a separate project with its own .claude/CLAUDE.md. The web application's project CLAUDE.md enforces strict security conventions; the internal tool's project CLAUDE.md has appropriate relaxed conventions. Project-level configuration scopes standards to their relevant context.

Question 8

Flag question

Correct Mark 1.00 out of 1.00

A developer creates a skill that generates database migrations. They want to ensure the skill can only read existing migration files for reference — it should never write, modify, or execute anything during analysis. How is this restriction configured?

- a. Add a system prompt instruction: 'Do not write or execute files during analysis.'
- b. Use context: fork to isolate the skill so it cannot affect the main session.
- c. Remove Write and Bash from the project's global allowed-tools list.
- d. Set allowed-tools: ['Read', 'Grep', 'Glob'] in the SKILL.md frontmatter — this restricts the skill's tool access to read-only operations at the configuration level, making it impossible to accidentally write or execute files regardless of what the model attempts. ✓

The correct answer is: Set allowed-tools: ['Read', 'Grep', 'Glob'] in the SKILL.md frontmatter — this restricts the skill's tool access to read-only operations at the configuration level, making it impossible to accidentally write or execute files regardless of what the model attempts.

Question 9

Flag question

Correct Mark 1.00 out of 1.00

Your team adopts a new API versioning convention: all new endpoints must use /v2/ prefix. This convention must apply immediately to all 15 developers without requiring individual action. What is the deployment mechanism?

a. Create a /check-api-version slash command developers invoke before each commit.
b. Email the 15 developers with the new convention.
c. Add the /v2/ prefix convention to the project-level .claude/CLAUDE.md and commit it to the main branch. All 15 developers receive the convention automatically on their next git pull — no individual action required. ✓
d. Add the convention to each developer's ~/.claude/CLAUDE.md individually.

The correct answer is: Add the /v2/ prefix convention to the project-level .claude/CLAUDE.md and commit it to the main branch. All 15 developers receive the convention automatically on their next git pull — no individual action required.

Question 10

Flag question

Correct Mark 1.00 out of 1.00

Your CI pipeline invokes Claude Code but the process hangs indefinitely waiting for user input, causing the pipeline to time out. What is the correct fix?

a. Use the -p (--print) flag: claude -p 'Review PR'. This enables non-interactive print mode — Claude Code processes the prompt, outputs the result to stdout, and exits without waiting for interactive input. This is the documented correct approach for CI/CD integration. ✓
b. Add --timeout 60 to limit how long Claude Code waits for input.
c. Set the environment variable CLAUDE_CI=true before running Claude Code.
d. Redirect stdin to /dev/null: claude 'Review PR' < /dev/null

The correct answer is: Use the -p (--print) flag: claude -p 'Review PR'. This enables non-interactive print mode — Claude Code processes the prompt, outputs the result to stdout, and exits without waiting for interactive input. This is the documented correct approach for CI/CD integration.

Question 11

Flag question

Correct Mark 1.00 out of 1.00

Your team's Terraform engineers report that when editing .tf files, Claude Code loads React frontend conventions from the root CLAUDE.md, consuming irrelevant context tokens. The root CLAUDE.md contains all team conventions in a single file. What is the most efficient fix?

a. Remove React conventions from the root CLAUDE.md since Terraform engineers do not need them.
b. Migrate all conventions to .claude/rules/ files with YAML frontmatter path patterns. The Terraform rules file would have paths: ['**/*.tf', '**/*.tfvars'] so it only loads for Terraform files, and the React rules file would only load for .tsx/.jsx files — eliminating cross-technology context pollution. ✓
c. Instruct engineers to manually disable irrelevant convention sections before editing.
d. Create a separate CLAUDE.md in the terraform/ directory with only Terraform rules.

The correct answer is: Migrate all conventions to .claude/rules/ files with YAML frontmatter path patterns. The Terraform rules file would have paths: ['**/*.tf', '**/*.tfvars'] so it only loads for Terraform files, and the React rules file would only load for .tsx/.jsx files — eliminating cross-technology context pollution.

Question 12

Flag question

Correct Mark 1.00 out of 1.00

Your CI pipeline runs Claude Code in non-interactive mode. A developer asks: 'Why do we use -p instead of just piping the prompt through stdin?' What is the correct explanation?

a. -p runs faster because it uses a more efficient API endpoint.
b. Piping through stdin is equivalent — -p is just a shorter alternative.
c. -p (--print) is the documented non-interactive mode that disables the interactive REPL and outputs results to stdout with proper exit codes. Piping stdin is a Unix workaround that may cause unexpected behaviour and is not the correct Claude Code mechanism for CI/CD integration. ✓
d. Piping through stdin requires additional shell configuration that -p handles automatically.

The correct answer is: -p (--print) is the documented non-interactive mode that disables the interactive REPL and outputs results to stdout with proper exit codes. Piping stdin is a Unix workaround that may cause unexpected behaviour and is not the correct Claude Code mechanism for CI/CD integration.

Question 13

Flag question

Correct Mark 1.00 out of 1.00

Your team's CI pipeline uses Claude Code to review PRs and post structured findings as inline comments. The current output is free-form prose that the comment-posting script cannot parse reliably. What CLI configuration produces machine-parseable output?

a. Add 'output your findings as JSON' to the review prompt.
b. Pipe Claude Code output through jq to extract structured data.
c. Use claude -p with --output-format json and --json-schema pointing to a findings schema definition — this enforces structured, schema-validated JSON output that the comment-posting script can parse reliably without fragile text processing. ✓
d. Add --structured to the Claude Code CLI invocation.

The correct answer is: Use claude -p with --output-format json and --json-schema pointing to a findings schema definition — this enforces structured, schema-validated JSON output that the comment-posting script can parse reliably without fragile text processing.

Question 14

Flag question

Correct Mark 1.00 out of 1.00

You want to test whether a new coding convention actually improves code quality before rolling it out to the whole team. How should you test this change using Claude Code session management?

a. Create a separate test repository with the new convention applied.
b. Use fork_session from a recent development session baseline to create an isolated branch. Apply the new convention in the fork and test its impact there. The main session and other developers are unaffected until the convention is validated and committed to the shared CLAUDE.md. ✓
c. Ask one developer to test the convention on their machine for a week.
d. Apply the convention globally in CLAUDE.md and revert if it does not work.

The correct answer is: Use fork_session from a recent development session baseline to create an isolated branch. Apply the new convention in the fork and test its impact there. The main session and other developers are unaffected until the convention is validated and committed to the shared CLAUDE.md.

Question 15

Flag question

Correct Mark 1.00 out of 1.00

You are using Claude Code to implement a feature that requires changes across a React frontend, a Python API, and Terraform infrastructure. The task involves architectural decisions that affect all three layers. Which Claude Code mode should you use at the start?

a. Direct execution — architectural knowledge comes from doing, not planning.
b. Plan Mode for the Terraform layer only; direct execution for frontend and API.
c. Direct execution with the full codebase in context so Claude understands all dependencies.
d. Plan Mode — this task involves multi-layer changes, architectural decisions affecting multiple systems, and potential approaches that should be evaluated before implementing. Plan Mode enables codebase exploration and design review before any changes are committed. ✓

The correct answer is: Plan Mode — this task involves multi-layer changes, architectural decisions affecting multiple systems, and potential approaches that should be evaluated before implementing. Plan Mode enables codebase exploration and design review before any changes are committed.

Question 16

Flag question

Correct Mark 1.00 out of 1.00

Your team discovers that Claude Code generates slightly different code style across different developers' machines despite having a shared project CLAUDE.md. Investigation reveals each developer also has personal ~/.claude/CLAUDE.md files with different style preferences. How should this be resolved?

a. Accept style variation as an inherent property of AI-generated code.
b. Run a code formatter as a post-processing step to enforce style consistency.
c. Update the project-level .claude/CLAUDE.md with explicit, authoritative style rules that override any personal preferences for project code. Personal preferences can remain for non-project activities but project standards should be specific enough to take precedence for project work. ✓
d. Remove all personal ~/.claude/CLAUDE.md files to force uniform behaviour.

The correct answer is: Update the project-level .claude/CLAUDE.md with explicit, authoritative style rules that override any personal preferences for project code. Personal preferences can remain for non-project activities but project standards should be specific enough to take precedence for project work.

Question 17

Flag question

Correct Mark 1.00 out of 1.00

Your CI pipeline generates test files using Claude Code. After 3 months, developers notice that generated tests duplicate existing test scenarios, wasting review time. What is the root cause and fix?

a. Run a post-generation deduplication script to remove duplicate tests.
b. Claude Code's test generation capability has degraded — upgrade to a newer version.
c. The test generation invocation does not include existing test files in context. Without visibility into what already exists, Claude Code cannot avoid generating duplicate scenarios. Include existing test files in the context when invoking test generation so Claude Code can identify and avoid already-covered scenarios. ✓
d. The CI pipeline is using cached prompts from the initial deployment.

The correct answer is: The test generation invocation does not include existing test files in context. Without visibility into what already exists, Claude Code cannot avoid generating duplicate scenarios. Include existing test files in the context when invoking test generation so Claude Code can identify and avoid already-covered scenarios.

Question 18

Flag question

Correct Mark 1.00 out of 1.00

Your development team has a convention: all database queries must use parameterised statements. This must apply globally to all developers. After adding it to .claude/CLAUDE.md, a developer bypasses it saying 'Claude kept suggesting parameterised queries and I told it to use string concatenation instead.' What does this reveal about prompt-based conventions?

a. This reveals the fundamental limitation of prompt-based conventions: they are probabilistic and can be overridden by user instructions in the session. For compliance-critical conventions that must have zero exceptions, implement linting rules or hooks that enforce the convention at the tool or CI level, not just in the prompt. ✓
b. Remove the convention from CLAUDE.md since it cannot be reliably enforced.
c. Add 'This convention cannot be overridden' to the CLAUDE.md entry.
d. The developer violated the convention — enforce it through code review.

The correct answer is: This reveals the fundamental limitation of prompt-based conventions: they are probabilistic and can be overridden by user instructions in the session. For compliance-critical conventions that must have zero exceptions, implement linting rules or hooks that enforce the convention at the tool or CI level, not just in the prompt.

Question 19

Flag question

Correct Mark 1.00 out of 1.00

A new security standard requires that all API responses include a Content-Security-Policy header. This standard must be applied immediately across all developers and all files. What is the fastest correct deployment mechanism?

a. Create a new /security-check command that developers invoke before each commit.
b. Update each developer's personal ~/.claude/CLAUDE.md individually.
c. Email all developers with the new requirement and ask them to remember it.
d. Add the Content-Security-Policy requirement to the project-level .claude/CLAUDE.md and commit it to version control. All developers receive the standard automatically on their next pull — no individual action required. ✓

The correct answer is: Add the Content-Security-Policy requirement to the project-level .claude/CLAUDE.md and commit it to version control. All developers receive the standard automatically on their next pull — no individual action required.

Question 20

Flag question

Correct Mark 1.00 out of 1.00

You have a complex legacy codebase investigation session that has been running for 6 hours. The context is heavily loaded with tool outputs. A colleague suggests starting fresh every time to avoid context issues. What is the correct nuanced position?

a. Always start fresh — context degradation makes long sessions unreliable.
b. Never restart — context continuity always outweighs context quality.
c. Restart every 3 hours as a standard practice regardless of context quality.
d. Long sessions with accumulated verbose tool output should use /compact to manage context. For genuinely stale content (modified files, obsolete findings), starting fresh with a structured summary is appropriate. The decision depends on the nature of what's in context — verbose exploration output vs. valuable established findings — not on session duration alone. ✓

The correct answer is: Long sessions with accumulated verbose tool output should use /compact to manage context. For genuinely stale content (modified files, obsolete findings), starting fresh with a structured summary is appropriate. The decision depends on the nature of what's in context — verbose exploration output vs. valuable established findings — not on session duration alone.

Previous activity
Tool Design & MCP Integration - Practice Assessment

Jump to...

Next activity
Prompt Engineering & Structured Output - Practice Assessment